UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INFORMÁTICA CULIACÁN

MANUAL DE PRÁCTICAS DE MONGODB

COLECCIÓN, CONSULTA, ACTUALIZACIÓN Y ELIMINACIÓN DE DOCUMENTOS CON MONGODB



ASIGNATURA: Bases de Datos Emergentes

PROGRAMA EDUCATIVO: Licenciatura en Ingeniería en Ciencia de Datos

RESPONSABLE: Dra. Lidia Yadira Pérez Aguilar

FECHA: agosto 2026

DATOS • CONOCIMIENTO • DECISIONES

Tecnología que transforma información en valor

PRÁCTICA 6. Aggregation Pipeline I

Objetivo

Utilizar el framework de agregación de MongoDB para filtrar, agrupar, resumir y ordenar información de la colección ventas, obteniendo indicadores básicos del comportamiento de ventas mediante $match, $group, $sum, $avg, $min, $max, $count y $sort.

Actividades

- Ejecutar el código en el orden indicado y verificar los resultados.
- Registrar evidencia de las consultas ejecutadas.
- Guardar el código en un documento .js
- Subirlo al repositorio GitHub

Entregable

- Coding completo y ejecutable.
- El un pdf con las respuestas a los ejercicios.

0. Preparar el entorno

- **Inicie MongoDB y posteriormente mongosh.**
- **Seleccione la base de datos utilizada en las prácticas anteriores**

```
use ventas_online
```

- **Compruebe las colecciones existentes:**

```
show collections
```

- **Contar los documentos**

```
db.productos.countDocuments()
```

Resultado esperado: 100

- **Contar las ventas**

```
db.ventas.countDocuments()
```

Resultado esperado: 100

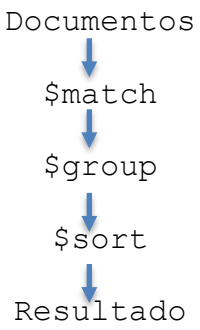
1. Introducción a aggregate()

Permite procesar documentos mediante una secuencia de etapas denominada pipeline. Hasta ahora hemos utilizado principalmente `find()` para recuperar documentos.

El método:

```
db.ventas.aggregate([
    ...
])
```

Cada etapa recibe los documentos de la etapa anterior, realiza una operación y envía el resultado a la siguiente. Podemos representarlo así:



2. Operador $group

Agrupa todos los documentos en un solo grupo.

Contar todas las ventas:

```
db.ventas.aggregate([
    {
        $group: {
            _id: null,
            numero_ventas: { $sum: 1 }
        }
    }
])
```

Nota:

- `_id:null` agrupa todos los documentos en un solo grupo.
- `$sum:1` suma una unidad por cada documento.

Resultado esperado:

```
{
    _id: null,
    numero_ventas: 100
}
```

3. Operador $sum

Suma los valores almacenados en el campo

Calcular ingresos totales (Sumar el campo total)

```
db.ventas.aggregate([
    {
        $group: {
            _id: null,
            ingreso_total: { $sum: "$total" }
        }
    }
])
```

Aquí `$sum:"$total"` suma los valores almacenados en el campo `total`.

4. Operador $avg

El operador $avg calcula el promedio de los valores.

Calcular ticket promedio

```
db.ventas.aggregate([
    {
        $group: {
            _id: null,
            ticket_promedio: { $avg: "$total" }
        }
    }
])
```

5. Varios indicadores en una sola agregación

Consultar: número de ventas; ingreso total; ticket promedio; venta de menor importe; venta de mayor importe.

```
db.ventas.aggregate([
    {
        $group: {
            _id: null,
            ventas: { $sum: 1 },
            ingreso_total: { $sum: "$total" },
            ticket_promedio: { $avg: "$total" },
            venta_minima: { $min: "$total" },
            venta_maxima: { $max: "$total" }
        }
    }
])
```

6. Agrupar por canal

Consultar cuántas ventas se realizaron por cada canal

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$canal",
            ventas: { $sum: 1 }
        }
    }
])
```

Aquí `_id:"$canal"` significa que los documentos se agruparán según el valor del campo `canal`.

7. Ingreso por canal

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$canal",
            ventas: { $sum: 1 },
            ingreso_total: { $sum: "$total" },
            ticket_promedio: { $avg: "$total" }
        }
    }
])
```

8. Ordenar resultados con $sort

Ordenar los canales de mayor a menor según ingreso:

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$canal",
            ventas: { $sum: 1 },
            ingreso_total: { $sum: "$total" }
        }
    },
    {
        $sort: {
            ingreso_total: -1
        }
    }
])
```

9. Agrupar por estado de la venta

Comparar las ventas según su estado: "completada", "pendiente" y "cancelada".

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$estado",
            ventas: { $sum: 1 },
            ingreso_total: { $sum: "$total" }
        }
    },
    {
        $sort: {
            ventas: -1
        }
    }
])
```

10. Filtrar antes de agrupar con $match

Analizar únicamente las ventas completadas:

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada"
        }
    },
    {
        $group: {
            _id: null,
            ventas_completadas: { $sum: 1 },
            ingreso: { $sum: "$total" },
            promedio: { $avg: "$total" }
        }
    }
])
```

Nota: Aquí aparece una idea importante

$match: selecciona documentos

$group: resume esos documentos

11. Filtrar y agrupar por canal

Mostrar las ventas completadas agrupadas por canal:

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada"
        }
    },
    {
```

```
        $group: {
            _id: "$canal",
            ventas: { $sum: 1 },
            ingreso: { $sum: "$total" }
        }
    },
    {
        $sort: {
            ingreso: -1
        }
    }
])
```

12. Agrupar por cliente

Determinar cuántas compras ha realizado cada cliente y cuánto ha gastado:

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$cliente_id",
            compras: { $sum: 1 },
            gasto_total: { $sum: "$total" },
            gasto_promedio: { $avg: "$total" }
        }
    },
    {
        $sort: {
            gasto_total: -1
        }
    }
])
```

13. Obtener los clientes con mayor gasto

Obtenemos el Top 5 de clientes según gasto acumulado.

Agregar `$limit` al pipeline:

```
db.ventas.aggregate([
    {
        $group: {
            _id: "$cliente_id",
            compras: { $sum: 1 },
            gasto_total: { $sum: "$total" }
        }
    },
    {
        $sort: {
            gasto_total: -1
        }
    },
    {
```

```
        $limit: 5
    }
])
```

14. Operador $count

Contar cuántas ventas están completadas:

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada"
        }
    },
    {
        $count: "ventas_completadas"
    }
])
```

Esto permite comparar $count con:

```
{ $sum: 1 }
```

15. Combinar varias etapas

Construir un pipeline completo:



Buscar ventas completadas con total mayor o igual a 1,000 pesos, agruparlas por canal y ordenar los resultados por ingreso.

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada",
            total: { $gte: 1000 }
        }
    },
    {
        $group: {
            _id: "$canal",
            ventas: { $sum: 1 },
            ingreso: { $sum: "$total" },
```

```
            promedio: { $avg: "$total" }
        }
    },
    {
        $sort: {
            ingreso: -1
        }
    }
])
```

Ejercicios de la Práctica 6

1. Calcular el número total de ventas utilizando $group y $sum.
2. Calcular el ingreso total generado por todas las ventas.
3. Calcular el importe promedio de las ventas.
4. Obtener en una sola consulta el número de ventas, ingreso total, venta mínima, venta máxima y ticket promedio.
5. Contar cuántas ventas existen por canal.
6. Calcular el ingreso total por canal.
7. Calcular el ticket promedio por canal.
8. Ordenar los canales desde el de mayor ingreso hasta el de menor.
9. Contar cuántas ventas existen por estado.
10. Calcular el ingreso asociado con cada estado.
11. Analizar únicamente las ventas cuyo estado sea "completada".
12. Calcular cuántas ventas completadas existen y cuál es su ingreso total.
13. Agrupar las ventas completadas por canal.
14. Calcular cuántas ventas ha realizado cada cliente.
15. Calcular el gasto acumulado por cliente.
16. Calcular el gasto promedio por cliente.
17. Obtener los 5 clientes con mayor gasto acumulado.
18. Obtener los 10 clientes con mayor número de compras.
19. Filtrar ventas con total mayor o igual a $2,000 y calcular cuántas existen por canal.
20. Diseñar un pipeline propio que incluya al menos $match, $group y $sort, y explicar qué indicador comercial obtiene.

- Sube a tu repositorio:
    - El archivo: Practica06-Aggregation-Pipeline-I.js
    - El archivo: Practica06-Aggregation-Pipeline-I.pdf (Con la portada)